TP Integrador 1 – Teoría para el coloquio

Redes de Datos II · LINTI–UNLP · Grupo 0P · Bloques asignados: **130.0.16.0/20** y **2002:dae1::/32**

Todo lo que se usó en los puntos obligatorios del TP, explicado desde cero: definición → regla → cómo se hace → ejemplo con nuestra red. Basado en las clases de la cátedra (IPv4, IPv6, ARP, ICMP) y en los ejemplos resueltos de la práctica. Archivo de la resolución: `tpi_1_estatico.imn` (emulador CORE).

Contenido

1. Antes de empezar: qué es cada cosa
2. El TP en una página
3. Direccionamiento IPv4 (VLSM)
4. Punto a: IPv6 en Net5 y Net6
5. Punto b: ruteo estático IPv4 e IPv6
6. Punto d: ping y traceroute
7. Punto e: captura n11 → n33 (ARP e ICMP)
8. Punto f: traceroute capturado
9. Punto h: IPv6 e ICMPv6
10. CORE y comandos
11. Errores conceptuales que hay que evitar

Cada punto tiene arriba un recuadro con lo que pide el enunciado y lo que tenés que saber explicar, y abajo las preguntas probables de ese punto.

1. Antes de empezar: qué es cada cosa

Esta sección no tiene nada del TP todavía. Explica con palabras simples los equipos y las ideas que aparecen en todo el resto. Si entendés esta página, el resto se lee mucho más fácil.

Los equipos

Equipo	Qué es	En el TP
Host	Una computadora "final": la que usa una persona o un servidor. Manda y recibe sus propios paquetes, pero no reenvía paquetes de otros.	n11, n12, n13, n14, n29 … n35 (los íconos de PC del dibujo)
Switch	Conecta varios equipos dentro de **una misma red local** (LAN). No mira direcciones IP; solo reparte tramas por MAC. No sabe nada de ruteo.	Los íconos chatos debajo de cada router de acceso (n3, n4, n19…)
Router	Conecta **redes distintas** entre sí. Mira la IP de destino de cada paquete y decide por qué cable mandarlo. Tiene una IP distinta en cada cable (interfaz).	Todos los íconos redondos: spines, leafs y routers de acceso
Router de acceso	El router "de la puerta" de cada LAN. Es la salida de esa red hacia el resto del mundo.	n1, n2, n17, n18, n21, n22, n23, n24
Leaf (hoja)	Router de nivel intermedio. Junta a varios routers de acceso y los conecta con el núcleo. Se llama "hoja" porque está en el borde del árbol, del lado de las redes de usuarios.	leaf1, leaf2, leaf3, leaf4 (uno por cada grupo Net1 … Net4)
Spine (columna)	Router del **núcleo**. Une a todos los leafs entre sí. Se llama "columna vertebral" porque todo el tráfico entre leafs pasa por él.	spine1 y spine2

La idea de spine-leaf, en simple

Pensalo como un edificio de oficinas. Cada piso tiene su **ascensor de piso** (leaf) que junta a las oficinas de ese piso (las redes Net1, Net2…). Los pisos no están conectados directamente entre sí. Para ir de un piso a otro siempre se pasa por el **hall central** (spine). Hay dos halls centrales para que, si uno se rompe, quede el otro.

Nivel 1 · Spines (núcleo): spine1 y spine2. Conectan a todos los leafs. No se conectan entre sí. De spine1 cuelga Net5 y de spine2 cuelga Net6.

▲ ▼

Nivel 2 · Leafs (distribución): leaf1 a leaf4. Cada uno conectado a los DOS spines y a sus dos routers de acceso. No se conectan entre sí.

▲ ▼

Nivel 3 · Routers de acceso: n1, n2, n17, n18, n21, n22, n23, n24. Cada uno es la puerta de salida de una LAN.

▲ ▼

Nivel 4 · LANs (switch + hosts): las redes de usuarios. Net1 a Net4 tienen dos LANs cada una; Net5 y Net6 cuelgan directo de los spines.

Figura A. Los cuatro niveles de la topología, de arriba (núcleo) hacia abajo (usuarios).

Para ir de un host de Net1 a uno de Net4, el paquete sube y baja por la escalera: **host → router de acceso → leaf1 → spine → leaf4 → router de acceso → host**. Nunca hay atajos entre leafs ni entre spines.

Ideas que se repiten en todo el TP

- **Red (o subred) / LAN:** un grupo de equipos que se pueden hablar directamente, sin pasar por un router. Todos tienen IPs que empiezan igual (el mismo "prefijo de red").
- **Interfaz:** cada "puerto" o placa de red de un equipo (eth0, eth1…). Cada interfaz tiene su propia IP. Un router con 4 cables tiene 4 IPs.
- **Enlace punto a punto:** un cable que une exactamente dos routers, sin switch. Como solo hay dos equipos, se le da una red diminuta de 2 IPs (/30).
- **Gateway (puerta de enlace):** el router al que un host le manda todo lo que no es de su propia red. Para n11 es n1 (130.0.18.1).
- **Dirección IP vs dirección MAC:** la IP dice **a quién va el paquete al final** (como la dirección escrita en un sobre). La MAC dice **a quién se lo entrego en este tramo** (como el cartero que lo pasa de mano en mano). La IP queda igual todo el viaje; la MAC cambia en cada salto.
- **Tabla de ruteo:** la lista de carteles que tiene cada router: "para ir a tal red, mandalo a tal vecino". El router la consulta para cada paquete.
- **Ruta por defecto (default):** el cartel "para cualquier otro lado, por acá". Se usa cuando ningún otro cartel coincide.
- **Next-hop (próximo salto):** la IP del vecino al que se le pasa el paquete. Tiene que estar conectado directamente al router.
- **Ruteo estático:** las rutas las escribe una persona a mano (es lo que hizo tu compañero). Lo opuesto es el ruteo dinámico, donde los routers se las enseñan solos con un protocolo.
- **Dual stack:** cada equipo tiene a la vez una IP versión 4 (130.0…) y una versión 6 (2002:dae1…), y las dos funcionan por separado.

> **Cómo viaja un paquete, en cuatro frases**
>
> 1) El host mira si el destino está en su propia red; si no, se lo da a su gateway.
> 2) Cada router mira la IP de destino, busca en su tabla el cartel que mejor coincide y se lo pasa al vecino que indica.
> 3) En cada tramo, el paquete viaja dentro de una "trama" con las MAC de ese tramo; para saber la MAC del vecino se usa ARP.
> 4) El último router ve que el destino está en una red suya y se lo entrega directo. La respuesta hace el mismo proceso al revés.

> **Cómo leer las IPs abreviadas de este documento**
>
> Cuando dice ".21.178" quiere decir 130.0.21.178: se omite el comienzo "130.0." porque es igual en todo el TP.

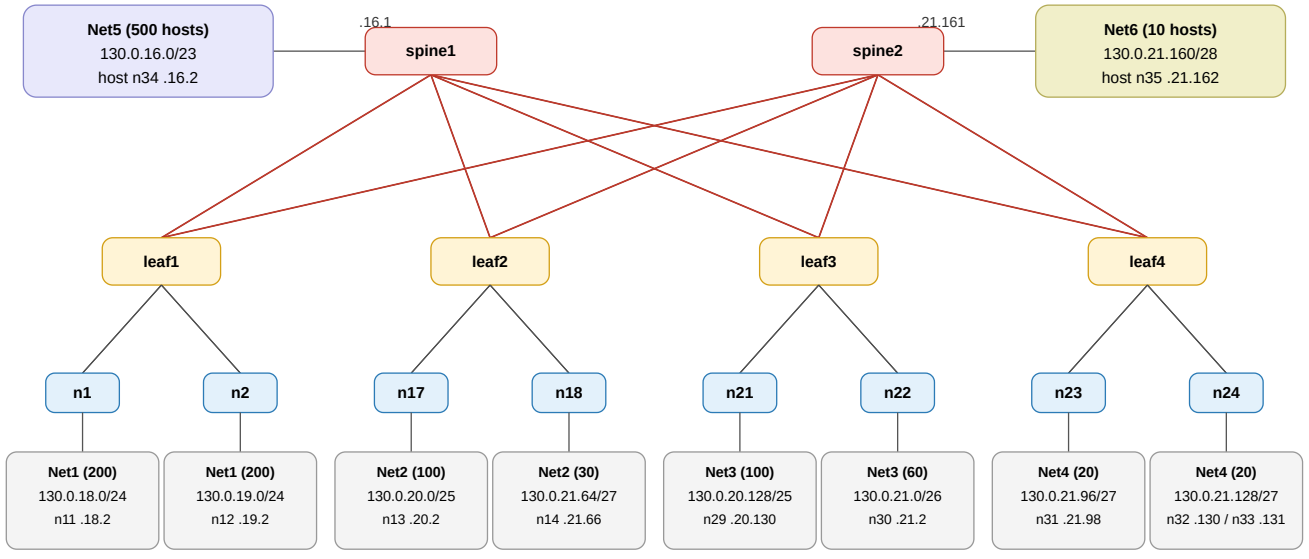
2. El TP en una página

> **Si ves números como /23, /24, /27 y no sabés qué son, no te preocupes**
> En esta página solo está el resumen. De dónde sale cada número se explica paso a paso en la sección 3 (Direccionamiento IPv4).

Qué pide el enunciado. Sobre la topología de la Figura 6 (dos spines, cuatro leafs, ocho routers de acceso y seis grupos de redes), direccionar con IPv4 usando el bloque asignado, con enlaces punto a punto /30 o /31 salvo donde hay switches. Después:

Punto	Qué pide	Estado
—	Direccionamiento IPv4 con VLSM	Hecho y verificado. Sección 3
a	IPv6 en Net5 y Net6, con conectividad IPv6 entre ambas	Hecho (IPv6 en toda la red). Sección 4
b	Ruteo estático IPv4 e IPv6 en toda la topología	Hecho y verificado. Sección 5
d	Probar con ping (ICMP) y traceroute	Hay que ejecutarlo. Sección 6
e	Capturar tráfico n11 → n33 y analizar ARP e ICMP	Hay que ejecutarlo. Sección 7
f	traceroute n11 → n33 capturando los mensajes	Hay que ejecutarlo. Sección 8
h	Probar conectividad IPv6 por separado y analizar ICMPv6	Hay que ejecutarlo. Sección 9

Los puntos c (NAT) y g (MTU) son opcionales, no se hicieron y esta guía no los cubre.



Enlaces rojos: leaf–spine (/30). Enlaces grises acceso–leaf: /30 desde 130.0.21.176. Gateway de cada LAN: la primera IP (.1, .65, .129…).
Cada leaf: sus dos LANs vía los routers de acceso · Net6 vía spine2 · todo lo demás (default) vía spine1.

Figura 1. Topología con el direccionamiento del TP. Los nombres son los del dibujo del enunciado.

> **Trampa al leer el .imn**
> En el archivo, cada nodo tiene un id interno (`node n1`) y un nombre (`hostname n34`). El que vale es el **hostname**, que es el que se ve en el dibujo. Ejemplo: `node n1` es el host n34 de Net5; `node n22` es n5-spine1.

3. Direccionamiento IPv4 (VLSM)

Qué pide el enunciado: "Para la figura 6, resolver el direccionamiento IPv4 con el bloque IP asignado. Considerar los enlaces punto a punto /30 o /31, salvo las redes donde hay switches/hubs." Net1: dos redes de 200 hosts · Net2: una de 30 y otra de 100 · Net3: una de 100 y otra de 60 · Net4: dos de 20 · Net5: una de 500 · Net6: una de 10.

Qué hizo el grupo / qué hay que mostrar: Hecho. Se repartió el bloque 130.0.16.0/20 con VLSM y se usaron /30 en los 16 enlaces entre routers.

Lo que tenés que poder explicar:

- Qué es una IP, una máscara y qué significa la barra (/24, /27).
- Cómo se calcula el tamaño de cada red y por qué se reparte de mayor a menor.
- La dirección de red, el broadcast y el rango usable de cualquier red del TP.
- Por qué los enlaces entre routers son /30 (y qué sería un /31).

3.1 IP: lo básico

Protocolo IP (RFC 791)

Protocolo de la capa de red. Es **no orientado a conexión** y de **mejor esfuerzo** (best effort): no garantiza entrega, orden ni ausencia de errores en los datos. Su unidad de datos (PDU) es el **datagrama** o paquete. Sus funciones: direccionamiento, ruteo/forwarding, multiplexación de protocolos superiores y fragmentación.

Capa de enlace vs capa de red. La capa de enlace (Ethernet, 802.11) lleva una **trama** entre dos nodos vecinos del mismo enlace, usando direcciones **MAC** (físicas, 48 bits). La capa de red lleva el **paquete** de origen a destino a través de muchas redes, usando direcciones **IP** (lógicas). Un paquete IP viaja dentro de una trama distinta en cada salto.

Campos del header IPv4 que hay que conocer

El header ocupa entre 20 y 60 bytes (20 sin opciones).

Campo	Bits	Para qué sirve
Version	4	4 para IPv4
Header Length (IHL)	4	Largo del header en palabras de 32 bits (5 = 20 bytes)
Total Length	16	Largo total del datagrama (header + datos), en bytes. Máximo 65.535
Identification	16	Igual en todos los fragmentos del mismo datagrama original
Flags	3	Reservado (0), **DF** (Don't Fragment), **MF** (More Fragments)
Fragment Offset	13	Posición del fragmento en el original, en unidades de 8 bytes
TTL	8	Se decrementa en 1 en cada router. Si llega a 0, se descarta. Evita loops
Protocol	8	Qué lleva adentro: 1 = ICMP, 6 = TCP, 17 = UDP
Header Checksum	16	Control de errores del header. Se recalcula en cada router (porque cambia el TTL)
Source / Destination Address	32 c/u	IP origen y destino

3.2 Direcciones, máscara y prefijo

Primero, la idea general: bloque y subredes

1) La cátedra le dio al grupo un **bloque**: 130.0.16.0/20. Es un "terreno" de 4096 direcciones (de 130.0.16.0 a 130.0.31.255). **Ningún equipo usa /20**: es solo el espacio que tenemos para repartir.
2) Ese bloque se **reparte** en redes más chicas (**subredes**), una por cada LAN y una por cada enlace entre routers. Cada subred tiene su propia barra (/23, /24, /27…) según cuántos equipos necesita. Cómo se hace el reparto está en el apartado 3.3.
3) Cada equipo usa la IP y la **barra de la subred donde está conectado**. Por ejemplo, n33 está en una LAN a la que le tocó una subred /27, por eso su IP se escribe 130.0.21.131**/27**.
En esta sección vemos qué significa la barra y cómo se calcula el rango de una subred. En el apartado 3.3, cómo se eligió cada una.

Dirección IP

Número de 32 bits que identifica una **interfaz** (no un equipo). Por eso un router tiene una IP por cada interfaz. Se escribe en notación decimal con puntos: 130.0.21.131.

Máscara de red / prefijo

Indica cuántos bits de la izquierda son la parte de **red**; el resto es la parte de **host**. Se escribe como máscara (255.255.255.224) o como largo de prefijo (/27). Son lo mismo: /27 = 27 unos seguidos de 5 ceros.

> **En criollo: qué significa la barra**
> Una IP son 32 bits (cuatro números de 0 a 255, cada uno de 8 bits). El número después de la barra dice cuántos de esos bits, contando desde la izquierda, son el "nombre de la red"; los que sobran numeran a los equipos dentro de esa red.
> · **/24**: los tres primeros números son la red y el último numera equipos. 130.0.18.0/24 = todas las IPs 130.0.18.*algo*.
> · **/23**: la red es un poco más grande: abarca dos "terceros números" seguidos (130.0.16.*x* y 130.0.17.*x*).
> · **/27**: la red es más chica: solo 32 IPs del último número (por ejemplo de .128 a .159).
> Cuanto **más grande** el número de la barra, **más chica** la red.

Regla formal: la red a la que pertenece una IP se obtiene con un **AND** bit a bit entre la IP y la máscara.

Direcciones de una subred

- **Dirección de red:** todos los bits de host en 0. No se asigna a nadie.
- **Broadcast dirigido:** todos los bits de host en 1. No se asigna a nadie.
- **Usables:** las del medio. Cantidad = $\mathbf{2^h - 2}$, con h = bits de host = 32 − prefijo.

> **Ejemplo con nuestra red: n33 = 130.0.21.131/27**
> Dato de partida: n33 está en la LAN de n24, y en el reparto (apartado 3.3) a esa LAN le tocó una subred /27. Ahora calculamos qué rango es:
> /27 → 32 − 27 = 5 bits de host → bloques de 2^5 = 32 direcciones. Los bloques en el último octeto empiezan en 0, 32, 64, 96, 128, 160… 131 cae en el bloque que empieza en 128.
> Red: **130.0.21.128** · Broadcast: **130.0.21.159** (128 + 32 − 1) · Usables: .129 a .158 (30 hosts). El gateway (n24) es .129.
> En binario, último octeto: 131 = 1000 0011, máscara 1110 0000 → AND = 1000 0000 = 128.

Truco del "número mágico": 256 − (valor de la máscara en el octeto donde corta) = tamaño del bloque. Para /27 la máscara es 255.255.255.224, y 256 − 224 = 32.

Prefijo	Máscara	Direcciones	Usables	Dónde se usó en el TP
/23	255.255.254.0	512	510	Net5 (500 hosts)
/24	255.255.255.0	256	254	Net1 (2 × 200)
/25	255.255.255.128	128	126	Net2 y Net3 (100)
/26	255.255.255.192	64	62	Net3 (60)
/27	255.255.255.224	32	30	Net2 (30), Net4 (2 × 20)
/28	255.255.255.240	16	14	Net6 (10)
/30	255.255.255.252	4	2	Enlaces punto a punto

Tipos y direcciones especiales

- **Unicast** (a una interfaz), **broadcast** (a todos los de una red), **multicast** (a un grupo, 224.0.0.0 a 239.255.255.255), **anycast** (al más cercano de un grupo; usa direcciones unicast).
- **Loopback:** 127.0.0.0/8 (127.0.0.1, localhost). Nunca sale a la red.
- **Broadcast limitado:** 255.255.255.255. **"Este host":** 0.0.0.0 (lo usa DHCP antes de tener IP).
- **Privadas (RFC 1918):** 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16. No se rutean en Internet (para salir a Internet un router tiene que traducirlas, eso es NAT).

130.0.0.0 es un rango **público** (sería clase B en el esquema de clases viejo). En el TP se usa como si fuera nuestro bloque público asignado.

3.3 Subnetting fijo y VLSM: nuestro reparto

> **Subnetting**
> Tomar bits de la parte de host de un bloque para crear subredes. Con n bits prestados salen 2^n subredes. La cátedra permite usar la primera y la última subred (la "subred cero").

> **Subnetting fijo**
> Todas las subredes con la misma máscara. Problema: la máscara tiene que alcanzar para la red más grande, y las chicas desperdician direcciones (un enlace de 2 hosts con un /23 sería absurdo).

> **VLSM (Variable Length Subnet Mask, RFC 1009)**
> Cada subred tiene la máscara que necesita. Así se aprovecha el bloque.

> **En criollo**
> Tenemos un terreno (el bloque 130.0.16.0/20) y hay que lotearlo para varias casas de distinto tamaño (las redes). VLSM es darle a cada casa un lote del tamaño justo. Se reparten primero los lotes grandes y después los chicos, para que no queden pedacitos sueltos que no le sirvan a nadie.

Procedimiento de VLSM (el de la cátedra)

1. Ordenar las redes de **mayor a menor** cantidad de hosts.
2. Para cada red, buscar el menor h tal que $2^h - 2 \geq$ hosts pedidos (contando el gateway, que también es una IP). El prefijo es $32 - h$.
3. Asignar la primera porción libre de ese tamaño, partiendo el espacio libre en mitades sucesivas. El resto queda libre para la siguiente.
4. Al final, los enlaces punto a punto (/30).

¿Por qué de mayor a menor? Porque cada subred tiene que empezar en un múltiplo de su tamaño (estar **alineada**). Si se asignan primero las chicas, quedan huecos que las grandes no pueden usar.

Nuestro cálculo

¿Alcanza el bloque? Sumando los tamaños: 512 + 256 + 256 + 128 + 128 + 64 + 32 + 32 + 32 + 16 + 16·4 = **1520** direcciones. El /20 asignado tiene 2^{12} = **4096**. Sobra (alcanzaba un /21 = 2048).

Paso 1: qué tamaño necesita cada red

Para cada red buscamos la potencia de 2 más chica que alcance para sus hosts, recordando que se pierden 2 direcciones (la de red y la de broadcast). Después, prefijo = 32 − bits de host.

Red	Hosts pedidos	Potencia de 2 que alcanza	Usables ($2^h - 2$)	Bits de host (h)	Prefijo (32 − h)
Net5	500	$512 = 2^9$ (256 no alcanza)	510	9	/23
Net1 a y b	200 c/u	$256 = 2^8$	254	8	/24
Net2 a, Net3 a	100 c/u	$128 = 2^7$	126	7	/25
Net3 b	60	$64 = 2^6$	62	6	/26
Net2 b	30	$32 = 2^5$	30	5	/27
Net4 a y b	20 c/u	$32 = 2^5$ (16 no alcanza)	30	5	/27
Net6	10	$16 = 2^4$	14	4	/28
Cada enlace	2	$4 = 2^2$	2	2	/30

Paso 2: repartir en orden, de la más grande a la más chica

Se empieza en la primera dirección del bloque (130.0.16.0). Cada red empieza donde terminó la anterior: **inicio siguiente = inicio actual + tamaño**. Si la suma pasa de 255 en el último número, se suma 1 al tercero (como contar con vuelto: 256 en el último número = 1 en el tercero).

Orden	Red	Tamaño	Empieza en	Cuenta	La siguiente empieza en
1	Net5	512	130.0.16.0	512 = 2 terceros números enteros (16 y 17)	130.0.18.0
2	Net1 a	256	130.0.18.0	256 = 1 tercer número entero	130.0.19.0
3	Net1 b	256	130.0.19.0	+1 en el tercero	130.0.20.0
4	Net2 a	128	130.0.20.0	0 + 128	130.0.20.128
5	Net3 a	128	130.0.20.128	128 + 128 = 256 → pasa al 21	130.0.21.0
6	Net3 b	64	130.0.21.0	0 + 64	130.0.21.64
7	Net2 b	32	130.0.21.64	64 + 32	130.0.21.96
8	Net4 a	32	130.0.21.96	96 + 32	130.0.21.128
9	Net4 b (n32, n33)	32	130.0.21.128	128 + 32	130.0.21.160
10	Net6	16	130.0.21.160	160 + 16	130.0.21.176
11–26	16 enlaces	4 c/u	130.0.21.176	176 + 16 × 4 = 240	130.0.21.240 (libre)

Así sale, por ejemplo, que la LAN de n33 es **130.0.21.128/27**: es la novena red del reparto, empieza en .128 y tiene 32 direcciones (de .128 a .159).

Paso 3: dentro de cada red, quién es quién

Convención usada en el TP: la **primera IP usable** es para el router (gateway) y la segunda para el host. Ejemplo Net4 b: red .128, gateway n24 = .129, n32 = .130, n33 = .131, broadcast .159.

El mismo reparto dibujado como árbol (cada red se parte en dos mitades hasta llegar al tamaño justo; es la forma en que lo muestra la cátedra):

```
130.0.16.0/20
 ├─ 130.0.16.0/21
 │   ├─ 130.0.16.0/22
 │   │   ├─ 130.0.16.0/23  → Net5 (500)
 │   │   └─ 130.0.18.0/23
 │   │       ├─ 130.0.18.0/24  → Net1 a (200)
 │   │       └─ 130.0.19.0/24  → Net1 b (200)
 │   └─ 130.0.20.0/22
 │       ├─ 130.0.20.0/23
 │       │   ├─ 130.0.20.0/24
 │       │   │   ├─ 130.0.20.0/25    → Net2 a (100)
 │       │   │   └─ 130.0.20.128/25  → Net3 a (100)
 │       │   └─ 130.0.21.0/24  (se sigue partiendo en mitades)
 │       │       ├─ 130.0.21.0/26    → Net3 b (60)
 │       │       ├─ 130.0.21.64/27   → Net2 b (30)
 │       │       ├─ 130.0.21.96/27   → Net4 a (20)
 │       │       ├─ 130.0.21.128/27  → Net4 b (20)
 │       │       ├─ 130.0.21.160/28  → Net6 (10)
 │       │       ├─ 130.0.21.176 a .239 → 16 enlaces /30
 │       │       └─ 130.0.21.240/28  libre
 │       └─ 130.0.22.0/23  libre
 └─ 130.0.24.0/21  libre
```

Red	Hosts	Prefijo	Rango usable	Broadcast	Gateway	Host del TP
Net5	500	130.0.16.0/23	.16.1 – .17.254	.17.255	.16.1 spine1	n34 .16.2
Net1 a	200	130.0.18.0/24	.18.1 – .18.254	.18.255	.18.1 n1	n11 .18.2
Net1 b	200	130.0.19.0/24	.19.1 – .19.254	.19.255	.19.1 n2	n12 .19.2
Net2 a	100	130.0.20.0/25	.20.1 – .20.126	.20.127	.20.1 n17	n13 .20.2
Net3 a	100	130.0.20.128/25	.20.129 – .20.254	.20.255	.20.129 n21	n29 .20.130
Net3 b	60	130.0.21.0/26	.21.1 – .21.62	.21.63	.21.1 n22	n30 .21.2
Net2 b	30	130.0.21.64/27	.21.65 – .21.94	.21.95	.21.65 n18	n14 .21.66
Net4 a	20	130.0.21.96/27	.21.97 – .21.126	.21.127	.21.97 n23	n31 .21.98
Net4 b	20	130.0.21.128/27	.21.129 – .21.158	.21.159	.21.129 n24	n32 .130, n33 .131
Net6	10	130.0.21.160/28	.21.161 – .21.174	.21.175	.21.161 spine2	n35 .21.162

> **Detalle fino: la red de 30 hosts**
>
> Net2 b pide 30 hosts y un /27 tiene exactamente 30 usables. Si contamos el gateway como una IP más, serían 31 y no entraría. Interpretación usual: "30 hosts" incluye todo lo conectado, o el enunciado ya contempla el router. Si te lo preguntan, decí que con el gateway serían 31 y que lo estricto sería un /26. Lo mismo, con más margen, para Net3 b (60 + 1 = 61 ≤ 62: entra).

3.4 Enlaces punto a punto: /30 y /31

Un enlace punto a punto conecta exactamente dos interfaces (dos routers), sin switch en el medio.

- **/30:** 4 direcciones: red, 2 usables, broadcast. Es lo clásico.
- **/31 (RFC 3021):** 2 direcciones, las dos usables. No hace falta dirección de red ni broadcast en un enlace de solo dos extremos. Ahorra la mitad.

En el TP se usaron **16 enlaces /30** (64 direcciones, de 130.0.21.176 a .239). Con /31 se hubieran usado 32. Están **agrupados por leaf** a propósito: los dos enlaces de acceso de cada leaf son contiguos, y eso permite sumarizarlos en los spines.

Enlace	Subred	Extremo A	Extremo B
n1 – leaf1	130.0.21.176/30	n1 .177	leaf1 .178
n2 – leaf1	130.0.21.180/30	n2 .181	leaf1 .182
n17 – leaf2	130.0.21.184/30	n17 .185	leaf2 .186
n18 – leaf2	130.0.21.188/30	n18 .189	leaf2 .190
n21 – leaf3	130.0.21.192/30	n21 .193	leaf3 .194
n22 – leaf3	130.0.21.196/30	n22 .197	leaf3 .198
n23 – leaf4	130.0.21.200/30	n23 .201	leaf4 .202
n24 – leaf4	130.0.21.204/30	n24 .205	leaf4 .206
leaf1 – spine1	130.0.21.208/30	leaf1 .209	spine1 .210
leaf1 – spine2	130.0.21.212/30	leaf1 .213	spine2 .214
leaf2 – spine1	130.0.21.216/30	leaf2 .217	spine1 .218
leaf2 – spine2	130.0.21.220/30	leaf2 .221	spine2 .222
leaf3 – spine1	130.0.21.224/30	leaf3 .225	spine1 .226
leaf3 – spine2	130.0.21.228/30	leaf3 .229	spine2 .230
leaf4 – spine1	130.0.21.232/30	leaf4 .233	spine1 .234
leaf4 – spine2	130.0.21.236/30	leaf4 .237	spine2 .238

Preguntas probables de este punto

1. **¿Qué bloque les asignaron y alcanza?**
   130.0.16.0/20, 4096 direcciones. Necesitamos 1520 (incluyendo 16 enlaces /30). Alcanza y sobra: con un /21 bastaba.
2. **¿Por qué Net5 es /23?**
   500 hosts + gateway no entran en un /24 (254 usables). Un /23 tiene 512 direcciones, 510 usables.
3. **¿Por qué se asigna de mayor a menor?**
   Para que cada subred quede alineada a su tamaño, sin huecos. Si se empieza por las chicas, las grandes no caen en un límite válido.
4. **Dirección de red, broadcast y rango de Net6.**
   130.0.21.160/28: red .160, broadcast .175, usables .161 a .174 (14).
5. **¿Por qué /30 en los enlaces? ¿Y /31?**

Un enlace punto a punto necesita 2 IPs; un /30 da 2 usables. El /31 (RFC 3021) también sirve y ahorra 2 direcciones por enlace porque no necesita red ni broadcast.

4. Punto a: IPv6 en Net5 y Net6

Qué pide el enunciado: "Configurar la red Net5 y Net6 con IPv6 con el bloque asignado y configurar para que exista conectividad IPv6 entre ambas."

Qué hizo el grupo / qué hay que mostrar: Hecho, y de más: se puso IPv6 en toda la red (dual stack) con el bloque 2002:dae1::/32. Para que Net5 y Net6 se hablen hacen falta las rutas IPv6 en los routers del medio, que están en el punto b.

Lo que tenés que poder explicar:

- Cómo se escribe y se abrevia una dirección IPv6.
- Por qué toda LAN IPv6 es /64.
- Cómo se partió el /32 asignado (/48 y después /64 por red).
- Qué es dual stack y por qué hubo que poner IPv6 también en los routers del medio.

> **En criollo**
> IPv6 es la versión nueva de IP. Hace lo mismo que IPv4 (darle dirección a cada interfaz y rutear paquetes), pero con direcciones mucho más largas, escritas en hexadecimal (números y letras de la a a la f) separadas por ":". En el TP las dos versiones conviven: cada equipo tiene una IPv4 y una IPv6, y el ruteo se configuró dos veces, una para cada una.

Por qué IPv6 y qué cambia

- Direcciones de **128 bits** ($2^{128} \approx 3{,}4 \times 10^{38}$).
- Header fijo de **40 bytes**, más simple: sin checksum, sin campos de fragmentación, sin opciones. Lo opcional va en **headers de extensión** encadenados con el campo Next Header.
- Campos renombrados: TTL → **Hop Limit**, Protocol → **Next Header**, TOS → **Traffic Class**. Nuevo: **Flow Label** (20 bits).
- No hay broadcast: se usa **multicast**. No hay ARP: se usa **NDP**.
- Fragmentación solo en el origen. Autoconfiguración (SLAAC).

Notación

- 8 grupos de 16 bits en hexadecimal, separados por ":".
- Se pueden omitir los ceros a la izquierda de cada grupo: 0db8 → db8.
- Una secuencia de grupos todos en cero se reemplaza por **"::"**, **una sola vez** por dirección.
- No hay máscara decimal: solo largo de prefijo (/64).

Ejemplo: `2002:dae1:0:3::2` = `2002:dae1:0000:0003:0000:0000:0000:0002`.

Tipos de direcciones

Tipo	Prefijo	Para qué
Global unicast	2000::/3	Públicas, como las IPv4 públicas. Las del TP (2002:dae1::/32) son de este rango
Link-local	fe80::/10 (se usa fe80::/64)	Solo válidas en el enlace. **Toda interfaz tiene una**, aunque no se configure. NDP las usa como origen
Unique Local (ULA)	fc00::/7 (en la práctica fd00::/8)	Equivalente a las privadas de IPv4
Multicast	ff00::/8	ff02::1 todos los nodos del enlace, ff02::2 todos los routers
Solicited-node multicast	ff02::1:ff00:0/104	Para NDP: ff02::1:ff + últimos 24 bits de la dirección
Loopback / no especificada	::1 / ::	localhost / "todavía no tengo dirección"
Documentación	2001:db8::/32	Ejemplos

Regla del /64

En IPv6 **toda LAN es un /64**: 64 bits de red y 64 bits de identificador de interfaz (IID), sin importar cuántos hosts tenga. SLAAC necesita 64 bits de IID para autogenerar direcciones. No se hace VLSM por cantidad de hosts: sobra espacio.

Plan del TP

Bloque asignado **2002:dae1::/32**. Se tomó el primer /48, **2002:dae1:0::/48**, y de ahí un /64 por red, variando el 4.º grupo (16 bits de subred → 65.536 subredes posibles). Cuentas: un /32 contiene 2^{16} = 65.536 /48, y un /48 contiene 65.536 /64.

Subred (/64)	Uso
2002:dae1:0:3::/64	Net5 (spine1 ::1, n34 ::2)
2002:dae1:0:4::/64	Net6 (spine2 ::1, n35 ::2)
…:5::, :7::, :2::, :9::	spine1 con leaf1, leaf2, leaf3, leaf4
…:6::, :8::, :1::, :a::	spine2 con leaf1, leaf2, leaf3, leaf4
…:10::, :11::, :1a::, :1b::	Net1: LANs de n1 y n2, enlaces n1–leaf1 y n2–leaf1
…:20::, :21::, :2a::, :2b::	Net2 (n17, n18 y sus enlaces)
…:30::, :31::, :3a::, :3b::	Net3 (n21, n22 y sus enlaces)
…:40::, :41::, :4a::, :4b::	Net4 (n23, n24 y sus enlaces)

En los enlaces punto a punto también se usó /64 (es lo recomendado para simplicidad; existe la opción /127, RFC 6164, equivalente al /31).

Dual stack

Cada interfaz tiene IPv4 e IPv6 a la vez (doble pila), y las dos funcionan de forma independiente: rutas, ARP/NDP, ping/ping6. Es la técnica de transición más común. Otras: túneles (6in4, GRE, **6to4**) y traducción.

Dato: 2002::/16 es el rango reservado para 6to4 (2002:WWXX:YYZZ:: embebe una IPv4; dae1 = 218.225). Como el bloque lo asignó la cátedra, no es un error; solo conviene saber qué es si preguntan.

Preguntas probables de este punto

1. **¿Por qué pusieron IPv6 en toda la red si el punto a pedía solo Net5 y Net6?**
   Para que Net5 y Net6 se comuniquen, todos los routers del camino tienen que rutear IPv6. Además el punto b pide ruteo IPv6 en toda la topología. Es dual stack.
2. **¿Por qué /64 en todas las redes IPv6?**
   Es el estándar para una LAN: 64 bits de identificador de interfaz, que necesita SLAAC. La cantidad de hosts no importa.
3. **¿Cómo dividieron el /32 de IPv6?**
   Tomamos el primer /48 (2002:dae1:0::/48) y un /64 por red variando el 4.º grupo, agrupando por leaf (:1x::, :2x::…) para sumarizar con /60.
4. **¿Qué es una dirección link-local?**
   fe80::/64, válida solo en el enlace, autogenerada en toda interfaz IPv6. NDP la usa como origen.
5. **¿Qué es 2002::/16?**
   El rango reservado para el mecanismo de transición 6to4. En el TP es simplemente el bloque que asignó la cátedra.

5. Punto b: ruteo estático IPv4 e IPv6

Qué pide el enunciado: "Resolver con ruteo estático la topología para IPv4 e IPv6."

Qué hizo el grupo / qué hay que mostrar: Hecho y verificado: cada host tiene ruta por defecto a su router, cada router de acceso a su leaf, cada leaf conoce sus LANs, manda Net6 a spine2 y el resto a spine1, y los spines conocen todo (con rutas sumarizadas). Lo mismo para IPv6.

Lo que tenés que poder explicar:

- Qué es una tabla de ruteo, una ruta por defecto y el next-hop.
- La regla del prefijo más largo (longest prefix match).
- Qué es sumarizar y dónde se hizo (130.0.18.0/23 y los /60 de IPv6).
- El camino completo de un paquete, de ida y de vuelta.
- Qué pasa si se cae spine1.

5.1 Ruteo y tablas de ruteo

> **Ruteo (routing)**
> Decidir el próximo salto (next-hop) y la interfaz de salida para un paquete. Lo hacen hosts y routers, mirando su tabla de ruteo. Es el plano de **control**.

> **Forwarding (despacho)**
> Pasar un paquete que entró por una interfaz hacia otra. Solo lo hacen los routers. Es el plano de **datos**. En Linux, un equipo reenvía solo si tiene `net.ipv4.ip_forward = 1`; en CORE eso lo activa el servicio **IPForward**.

> **En criollo**
> Cada router es como un cruce de rutas con carteles. Cuando llega un paquete, el router lee la dirección de destino, busca el cartel que coincide y lo manda por esa salida. No conoce el camino completo: solo sabe a qué vecino pasárselo. Si cada router del camino tiene bien sus carteles, el paquete llega.

Tabla de ruteo

Cada entrada tiene: **red destino**, **máscara/prefijo**, **next-hop** (o "directamente conectada"), **interfaz de salida** y **métrica**.

- **Rutas directamente conectadas:** aparecen solas al ponerle IP a una interfaz. El router ya sabe llegar a esa red.
- **Rutas estáticas:** las carga el administrador a mano: `ip route add <red>/<prefijo> via <next-hop>`.
- **Rutas dinámicas:** las aprende un protocolo de ruteo (RIP, OSPF, BGP). No se usaron en este TP (por eso el archivo se llama "estático").
- **Ruta por defecto:** `ip route add default via <next-hop>`. Se usa si nada más coincide.

> **Regla de oro: longest prefix match**
> Si varias entradas coinciden con el destino, el router usa la de **prefijo más largo** (la más específica). Ejemplo en leaf1: un paquete a 130.0.21.162 coincide con `130.0.21.160/28` (vía spine2) y con `default` (vía spine1). Gana /28 → va por spine2.

> **Regla del next-hop**
> El next-hop tiene que ser una IP de un **vecino directo**, es decir, estar en una red directamente conectada. Si no, el router no sabe por qué interfaz mandarlo ni a quién hacerle ARP.

> **Regla de ida y vuelta**
> Un ping funciona solo si el Echo Request llega **y** el Echo Reply vuelve. Cada router del camino de vuelta también necesita una ruta hacia la red de origen. Muchos errores de los TPs son rutas que funcionan de ida pero no de vuelta.

Host vs router

Un **host** tiene una tabla mínima: su red (conectada) y una default a su gateway. No reenvía paquetes ajenos. Un **router** tiene varias interfaces y reenvía. Un **host multihomed** tiene varias interfaces pero no rutea.

5.2 CIDR y sumarización

> **CIDR (Classless Inter-Domain Routing, 1993)**
> Elimina las clases A/B/C: toda red se describe con dirección + prefijo, de cualquier largo. Permite **agregar** (sumarizar, o "supernetting") varias redes en un prefijo más corto. Su objetivo principal es **achicar las tablas de ruteo**.

> **En criollo**
> Sumarizar es reemplazar varios carteles por uno solo que los abarca. En vez de que spine1 tenga "130.0.18.x → leaf1" y "130.0.19.x → leaf1", tiene un único cartel "130.0.18.x y 130.0.19.x → leaf1" (130.0.18.0/23). Se puede hacer solo si las redes están pegadas, forman un bloque "redondo" y van al mismo vecino.

Condiciones para sumarizar varias rutas en una

1. Las redes son **contiguas** y juntas forman un bloque **alineado** (la primera es múltiplo del tamaño total). En la práctica: comparten todos los bits hasta el nuevo prefijo.
2. Todas tienen el **mismo next-hop y la misma interfaz de salida** (regla de la cátedra). Si no, no se sumariza.
3. El sumario no debe cubrir redes que estén en otro lado. Si no, se manda tráfico al lugar equivocado.

Método: escribir en binario y contar los bits en común

```
130.0.18.0 → 3er octeto 0001 0010
130.0.19.0 → 3er octeto 0001 0011
                        ^^^^ ^^^   7 bits iguales en el 3er octeto
16 + 7 = 23 bits en común  →  sumario 130.0.18.0/23
```

```
130.0.21.176 → 4to octeto 1011 0000
130.0.21.180 → 4to octeto 1011 0100
                          ^^^^ ^     5 bits iguales
24 + 5 = 29  →  sumario 130.0.21.176/29  (cubre .176 a .183: los dos /30 de leaf1)
```

Dónde se sumarizó en el TP: en los spines. spine1 llega a todo lo de leaf1 con dos rutas: `130.0.18.0/23` (las dos LANs de Net1) y `130.0.21.176/29` (los dos enlaces de acceso). Lo mismo para los /30 de los otros leafs (/29 cada par).

Dónde no se pudo: las LANs de Net2 (130.0.20.0/25 y 130.0.21.64/27), Net3 (130.0.20.128/25 y 130.0.21.0/26) y Net4 (130.0.21.96/27 y 130.0.21.128/27) no forman bloques alineados. Cualquier prefijo que cubra las dos también cubriría redes de otro leaf. Por ejemplo, para Net4 el sumario sería 130.0.21.0/24 o mayor, que incluye Net3 b, Net2 b, Net6 y todos los enlaces. Por eso van por separado.

> **Ruta por defecto: la sumarización máxima**
> La ruta `default` es 0.0.0.0/0: cubre todo. Se usa cuando todas las demás redes no conectadas salen por el mismo next-hop. La cátedra lo muestra así: si todas las rutas de un router van al mismo vecino, se reemplazan por una default. **Solo puede haber una ruta default por tabla.**

5.3 Ruteo estático en nuestra topología

Qué es un spine y un leaf, en simple: ver sección 1.

> **Spine-leaf**
> Arquitectura de data center en dos niveles. Cada **leaf** se conecta a **todos** los **spines**. Los spines no se conectan entre sí y los leafs tampoco. Así, entre dos leafs cualesquiera siempre hay exactamente un spine en el medio, y hay tantos caminos alternativos como spines.

Lógica por tipo de equipo

Equipo	Rutas cargadas	Por qué
Hosts	default → su gateway	Todo lo que no es su LAN lo resuelve el router
Routers de acceso	default → su leaf	Solo tienen un camino hacia el resto
Leafs	sus 2 LANs → cada router de acceso Net6 → spine2 default → spine1	Net6 solo cuelga de spine2. Todo lo demás (incluida Net5 y los otros leafs) se alcanza por spine1
Spines	todas las redes de cada leaf → ese leaf (sumarizadas) la red del otro spine → vía leaf3 /30 del otro spine → vía el leaf de ese enlace sin default	El spine es el núcleo y debe conocer todo. Para llegar al otro spine tiene que pasar por un leaf

Tablas completas de ejemplo

n11 (host de Net1)

Destino	Next-hop	Interfaz	Origen
130.0.18.0/24	—	eth0	conectada
default	130.0.18.1	eth0	estática

n1 (router de acceso)

Destino	Next-hop	Interfaz	Origen
130.0.18.0/24	—	eth0	conectada
130.0.21.176/30	—	eth1	conectada
default	130.0.21.178 (leaf1)	eth1	estática

leaf1

Destino	Next-hop	Interfaz	Origen
130.0.21.176/30, .180/30, .208/30, .212/30	—	eth0, eth1, eth4, eth3	conectadas
130.0.18.0/24	130.0.21.177 (n1)	eth0	estática
130.0.19.0/24	130.0.21.181 (n2)	eth1	estática
130.0.21.160/28 (Net6)	130.0.21.214 (spine2)	eth3	estática
default	130.0.21.210 (spine1)	eth4	estática

leaf2, leaf3 y leaf4 son idénticos con sus propias redes.

spine1

Destino	Next-hop	Qué es
130.0.16.0/23, .21.208/30, .216/30, .224/30, .232/30	— (conectadas)	Net5 y sus 4 enlaces a leafs
130.0.18.0/23 · 130.0.21.176/29	.209 (leaf1)	Net1 sumarizada · enlaces de acceso de leaf1
130.0.20.0/25 · 130.0.21.64/27 · 130.0.21.184/29	.217 (leaf2)	Net2 · enlaces de leaf2
130.0.20.128/25 · 130.0.21.0/26 · 130.0.21.192/29	.225 (leaf3)	Net3 · enlaces de leaf3
130.0.21.96/27 · 130.0.21.128/27 · 130.0.21.200/29	.233 (leaf4)	Net4 · enlaces de leaf4
130.0.21.160/28	.225 (leaf3)	Net6 (leaf3 la pasa a spine2)
.21.212/30 · .220/30 · .228/30 · .236/30	.209 · .217 · .225 · .233	enlaces de spine2 con cada leaf

spine2 es simétrico: mismas rutas hacia los leafs (por sus propios enlaces) y Net5 vía leaf3 (.229).

Recorridos verificados (ida y vuelta)

- **n11 → n33:** n11 → n1 → leaf1 (default) → spine1 → leaf4 → n24 → n33. Vuelta: n24 → leaf4 (default) → spine1 → leaf1 (130.0.18.0/23) → n1 → n11. Simétrico.
- **n35 (Net6) → n34 (Net5):** n35 → spine2 → leaf3 (ruta Net5) → leaf3 default → spine1 → n34. Vuelta: spine1 → leaf3 (ruta Net6) → spine2 → n35.
- **n11 → n35:** n11 → n1 → leaf1 (ruta Net6 vía spine2) → spine2 → n35. Vuelta: spine2 → leaf1 (130.0.18.0/23) → n1 → n11.

> **Debilidad del diseño (pregunta probable)**
> La topología tiene caminos redundantes, pero el ruteo estático usa uno solo: casi todo pasa por spine1. Si spine1 se cae, se cae casi toda la red, y las rutas estáticas **no detectan fallas ni se reconfiguran solas**. Soluciones: ruteo dinámico (OSPF, o BGP en data centers) o ECMP (Equal-Cost Multi-Path: varias rutas iguales para repartir carga, por ejemplo `ip route add <red> nexthop via A nexthop via B`).

5.4 Ruteo IPv6 y sumarización /60

Sumarización /60 en los spines

Todo lo de leaf1 empieza con `:1x::`. El 4.º grupo es 0x0010 a 0x001f, que en binario comparte los primeros 12 bits (`0000 0000 0001`) y varía solo los últimos 4. Entonces 48 + 12 = **60 bits** en común: `2002:dae1:0:10::/60` cubre de :10:: a :1f::. Por eso cada spine tiene **una sola ruta IPv6 por leaf**.

```
/sbin/ip -6 route add 2002:dae1:0:10::/60 via 2002:dae1:0:5::2    # en spine1, todo leaf1
```

Preguntas probables de este punto

1. **¿Qué es una topología spine-leaf?**
   Dos niveles: cada leaf conectado a todos los spines; ni spines ni leafs entre sí. Entre dos leafs siempre hay un solo spine en el medio y caminos redundantes.
2. **¿Qué es la ruta por defecto y cuántas puede haber?**
   0.0.0.0/0, el prefijo más corto; se usa si ninguna otra coincide. Una sola por tabla.
3. **¿Cómo elige el router entre varias rutas?**
   Longest prefix match: gana la más específica.
4. **¿Qué es sumarizar y dónde lo hicieron?**
   Representar varias redes contiguas con un prefijo más corto para achicar la tabla. En los spines: 130.0.18.0/23 para las dos LANs de Net1, /29 para cada par de enlaces de acceso, y /60 por leaf en IPv6.
5. **¿Por qué no sumarizaron las redes de Net4?**
   130.0.21.96/27 y .128/27 no forman un bloque alineado; el menor prefijo que las cubre incluye redes de otros leafs.
6. **¿Por qué los leafs mandan Net6 a spine2 y el resto a spine1?**
   Net6 solo está conectada a spine2. Para todo lo demás alcanza una default a spine1, que conoce toda la red.
7. **¿Cómo llega spine1 a Net6 si no está conectado a spine2?**
   Vía leaf3, que tiene ruta a Net6 por spine2. Spine1 → leaf3 → spine2 → Net6.

8. ¿Qué pasa si se cae spine1?

Se pierde casi toda la conectividad, porque las rutas estáticas no se adaptan. Se resuelve con ruteo dinámico (OSPF/BGP) o ECMP.

9. ¿Qué hace el servicio IPForward?

Activa ip_forward para que el nodo reenvíe paquetes entre interfaces (actúe como router).

6. Punto d: ping y traceroute

Qué pide el enunciado: "Realizar test con ping (ICMP) y traceroute para probar que funciona la topología."

Qué hizo el grupo / qué hay que mostrar: Hay que ejecutarlo en CORE: pings entre hosts de distintas redes y un traceroute. Si todo responde, el direccionamiento y el ruteo están bien.

Lo que tenés que poder explicar:

- Qué es ICMP y qué mensajes usa ping (Echo Request 8 / Echo Reply 0).
- Cómo funciona traceroute (TTL creciente, Time Exceeded, Port Unreachable).
- Qué saltos debería mostrar el traceroute de n11 a n33.

> **ICMP (RFC 792)**
> Protocolo "helper" de IP para reportar errores y hacer consultas. Va encapsulado dentro de IP (Protocol = 1), pero no es de transporte. Es informativo: no corrige nada ni agrega confiabilidad.

Reglas: un mensaje de error ICMP incluye el header IP del datagrama que lo causó más al menos 8 bytes de sus datos. No se genera un error ICMP por otro error ICMP ni por paquetes a multicast.

Mensaje	Tipo	Código	Cuándo aparece
Echo Reply	0	0	Respuesta de ping
Destination Unreachable – Net Unreachable	3	0	El router no tiene ruta a esa red
Destination Unreachable – Host Unreachable	3	1	La red existe pero el host no responde (falla el ARP)
Destination Unreachable – Port Unreachable	3	3	Llegó al host pero el puerto UDP está cerrado (final del traceroute)
Echo Request	8	0	ping
Time Exceeded – TTL en tránsito	11	0	El TTL llegó a 0 en un router (traceroute)

ping (punto d)

Manda Echo Request (8) y espera Echo Reply (0). Mide el RTT y muestra el TTL con que volvió la respuesta (útil para estimar saltos: Linux arranca en 64).

traceroute (puntos d y f)

> **En criollo**
> traceroute sirve para ver **por qué routers pasa** un paquete hasta llegar al destino. El truco: manda paquetes que "mueren" a propósito después de 1 salto, después de 2, después de 3… Cada router donde muere un paquete avisa "se murió acá", y así se va armando la lista de routers del camino.

1. Manda un paquete con TTL = 1. El primer router lo descarta y devuelve **Time Exceeded (11/0)**: así se conoce su IP.
2. Repite con TTL = 2, 3, … Cada vez se revela un router más.

3. En Linux, traceroute manda **UDP** a puertos altos (desde 33434), que nadie usa. Cuando llega al destino, este responde **Port Unreachable (3/3)** y traceroute sabe que terminó.
4. Por defecto manda 3 sondas por salto (los 3 tiempos que muestra). `traceroute -I` usa Echo Request en vez de UDP; `-n` evita resolver nombres.

Salida esperada de `traceroute -n 130.0.21.131` desde n11 (deducida de las tablas; verificarla al correrla):

```
1  130.0.18.1     n1
2  130.0.21.178   leaf1
3  130.0.21.210   spine1
4  130.0.21.233   leaf4
5  130.0.21.205   n24
6  130.0.21.131   n33
```

La IP que aparece en cada salto es la de la interfaz del router que mira hacia n11, porque es la que usa como origen para mandar el ICMP de vuelta.

Preguntas probables de este punto

1. ¿Cómo funciona traceroute?
Envía sondas con TTL creciente. Cada router responde ICMP Time Exceeded (11/0) y el destino ICMP Port Unreachable (3/3), porque en Linux las sondas son UDP.

2. ¿Qué tipos ICMP vieron?
Echo Request 8, Echo Reply 0, Time Exceeded 11, Destination Unreachable 3 (código 3: puerto inalcanzable).

3. ¿Para qué sirve el TTL?
Evita que un paquete circule para siempre si hay un loop de ruteo. Traceroute lo aprovecha.

7. Punto e: captura n11 → n33 (ARP e ICMP)

Qué pide el enunciado: "Capturar puntualmente el tráfico de n11 hacia n33 y analizar: ARP e ICMP."

Qué hizo el grupo / qué hay que mostrar: Hay que ejecutarlo: vaciar la caché ARP, capturar con tcpdump o Wireshark en el enlace de n11 y hacer ping a n33.

Lo que tenés que poder explicar:

- Qué es ARP y para qué sirve (IP → MAC).
- Por qué n11 hace ARP por su gateway (130.0.18.1) y no por n33.
- Qué cambia y qué no en el paquete en cada salto (MAC sí, IP no).
- Qué mensajes aparecen en la captura y en qué orden.

ARP (RFC 826)

Protocolo "helper" de IP que traduce una **dirección IP** (lógica, 32 bits) a una **dirección MAC** (física, 48 bits) dentro de la misma red local. Se encapsula directo en la trama Ethernet (no va dentro de IP). Funciona solo, sin configuración.

Regla fundamental

ARP solo funciona **dentro de un dominio de broadcast** (la misma LAN o el mismo enlace). No se puede averiguar con ARP la MAC de un equipo de otra red: el router no reenvía broadcasts.

En criollo

Para mandar algo por el cable, el equipo necesita la MAC (la "dirección física") del vecino, pero solo conoce su IP. ARP es gritar en la red local: "¿quién tiene la IP 130.0.18.1? Decime tu MAC". El dueño de esa IP contesta solo a quien preguntó. Después se anota la respuesta para no volver a preguntar.

Funcionamiento

1. El emisor quiere mandar un paquete IP y necesita la MAC destino para armar la trama.
2. Mira su **tabla ARP** (caché). Si está la entrada, la usa.
3. Si no está, manda un **ARP Request** en **broadcast** (MAC destino ff:ff:ff:ff:ff:ff): "¿Quién tiene la IP X? Respondele a la IP Y, MAC Z".
4. Todos lo reciben, pero solo responde el que tiene la IP X, con un **ARP Reply unicast**: "X está en la MAC W".
5. El emisor guarda la entrada en la caché (con un tiempo de vida) y manda la trama.

¿Por qué IP pregunta? Los dos casos

- **Destino en mi misma red** (lo decide el emisor con su máscara): ARP por la IP del destino.
- **Destino en otra red**: ARP por la IP de mi **gateway** (el next-hop de la tabla de ruteo). La trama va a la MAC del router, aunque la IP destino del paquete sea la del destino final.

n11 → n33 paso a paso (punto e)

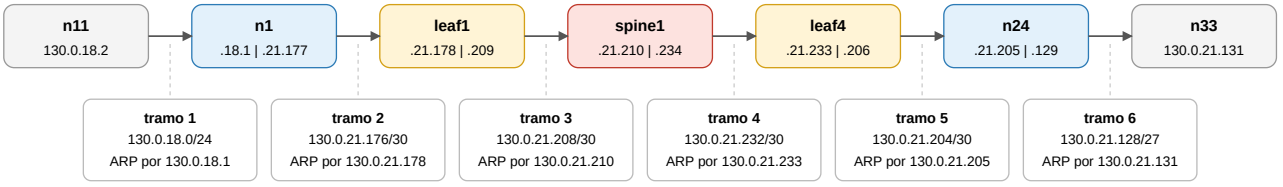


Figura 2. Camino de n11 a n33: por quién se hace ARP en cada tramo.

1. n11 quiere hacer ping a 130.0.21.131. AND con su máscara: 130.0.21.131 no está en 130.0.18.0/24, así que el paquete va a su default gateway 130.0.18.1.
2. n11 manda un ARP Request broadcast "¿quién tiene 130.0.18.1?". n1 responde con su MAC.
3. n11 manda la trama: MAC origen n11, MAC destino n1 · IP origen 130.0.18.2, IP destino 130.0.21.131 · ICMP Echo Request.
4. n1 recibe, mira su tabla (default vía .178), baja el TTL, recalcula el checksum, hace ARP por 130.0.21.178 (si no lo tiene) y arma una trama nueva hacia leaf1.
5. Lo mismo en cada salto, hasta que n24 ve que 130.0.21.131 está en su red conectada y hace ARP por la IP de n33.

Tramo	MAC origen	MAC destino	IP origen	IP destino
n11 → n1	n11 eth0	n1 eth0	130.0.18.2	130.0.21.131
n1 → leaf1	n1 eth1	leaf1 eth0	130.0.18.2	130.0.21.131
leaf1 → spine1	leaf1 eth4	spine1 eth1	130.0.18.2	130.0.21.131
spine1 → leaf4	spine1 eth4	leaf4 eth3	130.0.18.2	130.0.21.131
leaf4 → n24	leaf4 eth1	n24 eth1	130.0.18.2	130.0.21.131
n24 → n33	n24 eth0	n33 eth0	130.0.18.2	130.0.21.131

Conclusión: las MAC cambian en cada salto (son locales a cada enlace); las IP de origen y destino no cambian (son de extremo a extremo).

Qué se ve en la captura en el enlace de n11: ARP Request (broadcast) → ARP Reply (unicast) → Echo Request → Echo Reply. Si la caché ya estaba cargada no hay ARP: hay que vaciarla antes con `ip neigh flush all`.

Variantes de ARP (teoría de la clase)

- **Proxy ARP:** un nodo responde un ARP Request en nombre de otro.
- **Gratuitous ARP:** ARP Reply en broadcast sin que nadie lo pida, para anunciar o actualizar el mapeo IP–MAC propio.
- **ARP Probe / Announcement:** al asignarse una IP, el nodo pregunta si alguien la usa (probe); si nadie responde, la reclama (announcement).

Preguntas probables de este punto

1. Si n11 le hace ping a n33, ¿por quién pregunta ARP?

Por 130.0.18.1, su gateway, porque n33 está en otra red. El ARP por 130.0.21.131 lo hace n24.

2. **¿Qué cambia en el paquete en cada salto?**
   MAC origen y destino (trama nueva), TTL (−1) y checksum del header IP. Las IPs no cambian.
3. **¿Qué se ve en la captura, en orden?**
   ARP Request de n11 (broadcast, "¿quién tiene 130.0.18.1?"), ARP Reply de n1 (unicast, con su MAC), ICMP Echo Request de 130.0.18.2 a 130.0.21.131 y el Echo Reply de vuelta.
4. **¿Por qué el ARP Request va a broadcast y el Reply no?**
   El que pregunta no sabe la MAC de nadie, así que pregunta a todos (ff:ff:ff:ff:ff:ff). El que responde ya sabe la MAC del que preguntó (venía en el Request), así que le contesta solo a él.

8. Punto f: traceroute capturado

Qué pide el enunciado: "Realizar un traceroute entre los mismos equipos (n11 → n33), capturar los mensajes."

Qué hizo el grupo / qué hay que mostrar: Hay que ejecutarlo: capturar en el enlace de n11 mientras se corre traceroute -n 130.0.21.131. La teoría de traceroute está en el punto d (sección 6).

Lo que tenés que poder explicar:

- Qué paquetes manda traceroute y con qué TTL.
- Qué mensaje ICMP devuelve cada router y cuál devuelve el destino.

Captura del traceroute (punto f)

En el enlace de n11 se ve: UDP con TTL 1 → ICMP Time Exceeded desde 130.0.18.1; UDP con TTL 2 → Time Exceeded desde 130.0.21.178; … ; UDP con TTL 6 → ICMP Port Unreachable desde 130.0.21.131. Además, al principio, el ARP por el gateway.

Preguntas probables de este punto

1. **¿Qué mensajes se ven en la captura del traceroute?**
   Sondas UDP con TTL 1, 2, 3… desde n11. Por cada una, un ICMP Time Exceeded (tipo 11) de cada router del camino. Al final, ICMP Port Unreachable (tipo 3, código 3) desde n33. Al principio puede aparecer el ARP por el gateway.
2. **¿Por qué el destino responde distinto que los routers?**
   Porque a n33 la sonda le llega con TTL todavía válido, así que la procesa: es UDP a un puerto cerrado y responde Port Unreachable. A los routers les llega con TTL 1, lo bajan a 0, la descartan y avisan con Time Exceeded.
3. **¿Cuántos saltos hay de n11 a n33?**
   Cinco routers (n1, leaf1, spine1, leaf4, n24) más el destino: 6 líneas en el traceroute.

9. Punto h: IPv6 e ICMPv6

Qué pide el enunciado: "Probar conectividad en las redes con IPv6 (por separado), capturar tráfico y analizar ICMPv6."

Qué hizo el grupo / qué hay que mostrar: Hay que ejecutarlo: ping6 y traceroute6 de n34 (Net5) a n35 (Net6) capturando el tráfico.

Lo que tenés que poder explicar:

- Qué es ICMPv6 y por qué es más importante que ICMP en IPv4.
- Que en IPv6 no hay ARP: lo reemplaza NDP (Neighbor Solicitation y Neighbor Advertisement).
- A qué dirección va el Neighbor Solicitation (multicast solicited-node).
- Los tipos ICMPv6 que aparecen: 128/129 (ping6), 135/136 (NDP), 3 (traceroute6).

ICMPv6 (RFC 4443)

Mucho más importante que ICMP en IPv4: además de errores y ping, transporta **NDP** (que reemplaza a ARP y descubre routers) y **MLD** (que reemplaza a IGMP). Si se bloquea ICMPv6, IPv6 deja de funcionar.

NDP – Neighbor Discovery Protocol (RFC 4861)

Mapea direcciones IPv6 a MAC (como ARP), descubre routers y prefijos, detecta direcciones duplicadas (DAD) y vecinos inalcanzables (NUD). Usa 5 mensajes ICMPv6.

Mensaje ICMPv6	Tipo	Uso
Destination Unreachable	1	Sin ruta, puerto cerrado (código 4, al final del traceroute6)
Packet Too Big	2	Paquete mayor que la MTU (PMTU)
Time Exceeded	3	Hop Limit en 0 (traceroute6)
Echo Request / Reply	128 / 129	ping6
Router Solicitation (RS)	133	Un host pide a los routers que se anuncien
Router Advertisement (RA)	134	El router anuncia prefijo y gateway (SLAAC)
Neighbor Solicitation (NS)	135	"¿Quién tiene esta IPv6?" (equivale al ARP Request). También para DAD
Neighbor Advertisement (NA)	136	"La tengo yo, mi MAC es…" (equivale al ARP Reply)
Redirect	137	Avisa de un next-hop mejor

NS / NA vs ARP: la diferencia clave

El NS **no va a broadcast** (IPv6 no tiene). Va a la **solicited-node multicast** de la dirección buscada: `ff02::1:ff` + los últimos 24 bits. Solo escuchan esa dirección los nodos cuyas IPs terminan igual, así que se molesta a muchos menos equipos que con un broadcast.

> **Ejemplo del TP**
> n34 quiere llegar a su gateway 2002:dae1:0:3::1. Últimos 24 bits: 00:0001 → NS a **ff02::1:ff00:1** (en Ethernet, MAC destino 33:33:ff:00:00:01). spine1 responde con un NA unicast con su MAC.

SLAAC (para saber, no se usó)

El router manda RAs con el prefijo /64; el host arma su dirección con prefijo + IID, hace DAD y toma al router como gateway. Los flags del RA indican si además se usa DHCPv6: M = 1 (direcciones por DHCPv6), O = 1 (otros datos, como DNS, por DHCPv6). En el TP todo es estático, por eso no aparecen RS/RA en las capturas.

Qué hacer y qué se ve (punto h)

```
# desde n34 (Net5) hacia n35 (Net6)
ip -6 neigh flush all
ping6 -c 3 2002:dae1:0:4::2
traceroute6 -n 2002:dae1:0:4::2
ip -6 neigh                       # caché de vecinos
tcpdump -i eth0 -n icmp6          # captura
```

En la captura: NS de n34 a ff02::1:ff00:1 → NA de spine1 → Echo Request (128) → Echo Reply (129). Es posible ver también NS/NA usando direcciones **link-local** (fe80::…) como origen. El traceroute6 esperado es spine1 → leaf3 → spine2 → n35 (camino de Net5 a Net6), con Time Exceeded (3) en cada salto.

Preguntas probables de este punto

1. **¿Qué reemplaza a ARP en IPv6?**
   NDP: Neighbor Solicitation (135) a la multicast solicited-node y Neighbor Advertisement (136) unicast.
2. **¿A qué dirección manda n34 el Neighbor Solicitation para encontrar a su gateway?**
   A la multicast solicited-node de 2002:dae1:0:3::1, que es ff02::1:ff00:1 (ff02::1:ff + los últimos 24 bits de la dirección buscada).
3. **¿Por qué no aparecen Router Advertisements en la captura?**
   Porque todo se configuró a mano (estático). Los RA los mandan los routers cuando se usa autoconfiguración (SLAAC), y acá no se activó.

10. CORE y comandos

CORE (Common Open Research Emulator) emula redes en Linux: cada nodo es un contenedor liviano con su propia pila de red. El archivo `.imn` describe la topología. Se abre con `core-gui tpi_1_estatico.imn` y se arranca con Start; doble click en un nodo abre su terminal.

Cómo leer un bloque del .imn

```
node n23 {                          ← id interno
    type router
    model router                    ← router (model host = host; type lanswitch = switch)
    network-config {
    hostname n1                     ← nombre real (el del dibujo)
    interface eth0
     ip address 130.0.18.1/24
     ipv6 address 2002:dae1:0:10::1/64
    interface eth1
     ip address 130.0.21.177/30
     ipv6 address 2002:dae1:0:1a::1/64
    }
    services {IPForward StaticRoute}  ← activa el forwarding y el script de rutas
    interface-peer {eth0 n21}         ← eth0 va al nodo n21 (el switch n3)
    interface-peer {eth1 n25}         ← eth1 va a n25 (leaf1)
    custom-config { … staticroute.sh
        /sbin/ip route add default via 130.0.21.178
        /sbin/ip -6 route add default via 2002:dae1:0:1a::2 }
}
```

Los hosts usan el servicio **DefaultRoute** (`defaultroute.sh`) en vez de StaticRoute. Los bloques `link` del final son solo los cables. `iconcoords`, `labelcoords` y `canvas` son posiciones en pantalla.

Comandos

Comando	Para qué
`ip addr` / `ip -6 addr`	Ver IPs de las interfaces
`ip route` / `ip -6 route`	Ver la tabla de ruteo
`ip route add 130.0.18.0/23 via 130.0.21.209`	Agregar ruta estática
`ip route add default via 130.0.18.1`	Agregar ruta por defecto
`ip route get 130.0.21.131`	Qué ruta usaría para ese destino
`ip neigh` / `ip neigh flush all`	Ver / vaciar la caché ARP (y NDP con -6)
`sysctl net.ipv4.ip_forward`	1 = el nodo rutea
`ping`, `ping6`, `traceroute`, `traceroute6`	Pruebas de conectividad
`tcpdump -i eth0 -n arp or icmp`	Capturar ARP e ICMP
`tcpdump -i eth0 -n -e`	Mostrar también las MAC (útil para el punto e)
`tcpdump -i eth0 -n -w captura.pcap`	Guardar la captura para abrir en Wireshark

11. Errores conceptuales que hay que evitar

- **"El host hace ARP por la IP del destino."** Solo si está en su misma red. Si no, hace ARP por su gateway.
- **"Las IP cambian en cada salto."** No: cambian las MAC. Las IP son de extremo a extremo.
- **"ARP en IPv6 usa broadcast."** No existe ARP ni broadcast en IPv6: NDP con multicast solicited-node.
- **"Un /24 tiene 256 hosts."** Tiene 256 direcciones y 254 usables.
- **"Para que haya ping alcanza con la ruta de ida."** También hace falta la de vuelta.
- **"El next-hop puede ser cualquier router del camino."** Tiene que ser un vecino directo.
- **"Sumarizar es juntar redes cualesquiera."** Tienen que ser contiguas, alineadas y con el mismo next-hop.
- **"En IPv6 hago VLSM según la cantidad de hosts."** No: toda LAN es /64.
- **"traceroute usa ICMP Echo."** En Linux usa UDP por defecto; en Windows (tracert), ICMP Echo.